UNIVERSIDADE FEDERAL DA PARAÍBA

Centro de Informática - Ciência da Computação

PROJETO DE ENGENHARIA DE SISTEMAS DISTRIBUÍDOS

PLANO DE TESTES PARA O PROJETO ‘desafIAr’

Equipe: Arthur Vieira
Augusto Miguel
Carlos Eduardo
Igor Wanderley
Joás Gomes
Kalil Teotonio

João Pessoa - PB

2026

Sumário

1. **Objetivo** 3
2. **Fases** 3
3. **Arquivos-alvo** 3
4. **Requisitos de qualidade** 3
5. **Decisões** 4
6. **Desafios** 4

Objetivo

TL;DR: Desenvolver uma estratégia de testes para o app em Django **desafIAr**, focando em processamento assíncrono, integração com IA e padrões de resiliência baseados em ADRs. O plano abrange testes unitários, de integração, de ponta a ponta (E2E) e de resiliência para garantir a geração de desafios com baixa latência e tratamento de falhas controlado.

Fases

- **Fase 1: Configuração de Testes Unitários** — Criar casos de teste para componentes individuais como models, utilitários e agentes de IA. Utilizar o TestCase do Django e realizar *mock* de APIs externas para isolar a lógica.
- **Fase 2: Testes de Integração** — Testar as interações entre componentes, como o envio de vídeos para a fila de tarefas e os fallbacks de carregamento de desafios. Utilizar TestClient para views e fixtures de Redis/PostgreSQL.
- **Fase 3: Testes de Ponta a Ponta (E2E)** — Simular fluxos completos do usuário, desde o envio da URL do YouTube até o consumo do desafio. Utilizar Selenium ou o cliente de teste do Django com serviços reais em Docker.
- **Fase 4: Testes de Resiliência e Performance** — Implementar simulações de *circuit breaker*, testes de carga para pools do Redis e cenários de falha conforme as ADRs. Utilizar Locust para performance e scripts customizados para engenharia de caos (*chaos engineering*).
- **Fase 5: Integração CI/CD** — Adicionar os testes ao GitHub Actions (ou similar), garantindo execuções automatizadas a cada push. Incluir relatórios de cobertura e linting (não confirmado).

Arquivos-alvo

- **tests.py** — Expandir com suítes de testes abrangentes.
- **models.py** — Testar validações dos models Vídeo e Desafio.
- **views.py** — Testar endpoints da API com seus respectivos fallbacks.
- **agentes.py** — Criar mocks das chamadas da API escolhida para transcrição e geração de desafios.
- **obter_titulo.py** — Testar a busca de títulos do YouTube com timeouts.
- **settings.py** — Garantir configurações de teste para Redis, Celery e Banco de Dados.
- **docker-compose.yml** — Utilizar para testes de integração com serviços reais.

Requisitos de qualidade

- **Execução:** Executar todos os testes visando uma cobertura superior a **80% dos casos**.
- **Cobertura:** Usar pytest-cov para relatórios de cobertura, verificando caminhos críticos como tarefas assíncronas e fallbacks.

- **Verificação Manual:** Enviar uma URL do YouTube em ambiente de teste e confirmar o carregamento do desafio em menos de **2s**.
- **Performance:** Validar latência de consumo de desafios < **100ms** no Redis e < **500ms** no Banco de Dados.
- **Resiliência:** Simular falhas na API e verificar se o *circuit breaker* de fato impede falhas em cascata.

Decisões

- Priorizar testes para funcionalidades baseadas nas ADRs: pool assíncrono do Redis (**ADR-01**), circuit breaker (**ADR-02**) e integração com Gemini (**ADR-03**).
- Utilizar **mocks** para APIs externas (Gemini, YouTube) para evitar limites de taxa (*rate limits*) e garantir testes determinísticos.
- Incluir tanto testes automatizados quanto manuais: automatizados para regressão e manuais para validação de UX.
- **Escopo:** Cobrir "caminhos felizes" (*happy paths*), casos de borda (URLs inválidas, pools vazios) e modos de falha (API fora do ar, Redis indisponível).

Desafios

- **Orçamento de API:** Chamadas ao Gemini em testes podem gerar custos;
- **Treinamento da Equipe:** Escrever testes significativos para garantir os contextos das ADRs requereu conhecimento novo.
- **Monitoramento:** Tentar integrar testes com logs para rastrear as taxas de sucesso de tarefas assíncronas.